Beyond Reward Correctness: Distribution-Driven Mechanisms in LLM Reinforcement Learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Reinforcement learning (RL) with verifiable rewards (RLVR) has been shown to be an effective post-training approach for Large Language Models (LLMs). Since RL optimizes the model for maximum rewards, it is widely believed that the reward must strongly correlate with task success. However, this does not fully explain why training is robust to noisy self-generated rewards in self-supervised RL processes. Motivated by recent findings that entropy minimization training includes a distribution-driven process, we hypothesize that self-supervised RL includes a similar mechanism because it also minimizes response entropy. We test this hypothesis with a design that isolates the success factor from the distribution factor, and results show that both are effective reward signals, providing evidence to support the existence of the distribution-driven mechanism. Our findings advance current understanding of LLM RL, providing explanations for recent observations and informing future research to account for the distribution factor in training.

1 Introduction

Reinforcement learning (RL) with verifiable rewards (RLVR) has emerged as an effective post-training approach for large language models (LLMs) Guo et al. [2025a], Zhang et al. [2025a]. Generally, such approaches reward the model based on the success of its action trajectories, then update model weights to encourage successful behaviors and suppress unsuccessful ones, effectively enhancing problem-solving capability Guo et al. [2025a], Littman [2015], Mnih et al. [2015], Silver et al. [2016]. The reward signal is believed to be the most critical factor whose quality directly affects model performance, as it is the ultimate objective the model is optimized for Kaelbling et al. [1996], Sutton and Barto [1998]. Accordingly, a great deal of recent work has focused on providing more effective reward signals in RLVR She et al. [2025], Wang et al. [2024], Yuan et al. [2025], Guo et al. [2025b].

Recently, it was found that RLVR-style post-training can be done in a self-supervised manner Zuo et al. [2025], Zhang et al. [2025c,b], Agarwal et al. [2025]. In particular, the model's own generations were curated as rewards for RL training, such as self-consistency of the answer Zuo et al. [2025], Zhang et al. [2025c], internal confidence Agarwal et al. [2025], or self-generated marks Yuan et al. [2024], Wu et al. [2025]. The effectiveness of this training style has been verified by many works, showing comparable performance to training with ground-truth labels Zuo et al. [2025], Zhang et al. [2025c]. It was believed that the key factor behind such processes is the model's self-evaluation Zuo et al. [2025], Yuan et al. [2024], Wu et al. [2025] and cognitive ability Gandhi et al. [2025]. On the other hand, the quality of such guidance signals is often significantly lower than the ground truth. For example, our experimental results show that the model can reach performance comparable to training with the ground-truth oracle (often only 1–2% lower) with only 40%–70% reward accuracy. This raises a fundamental question: why do such noisy rewards still effectively improve model performance? Concurrently, Yue et al. Yue et al. showed that RLVR does not expand the reasoning boundary of the base model — the set of problems solvable by the RL-trained model (at pass@k)

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

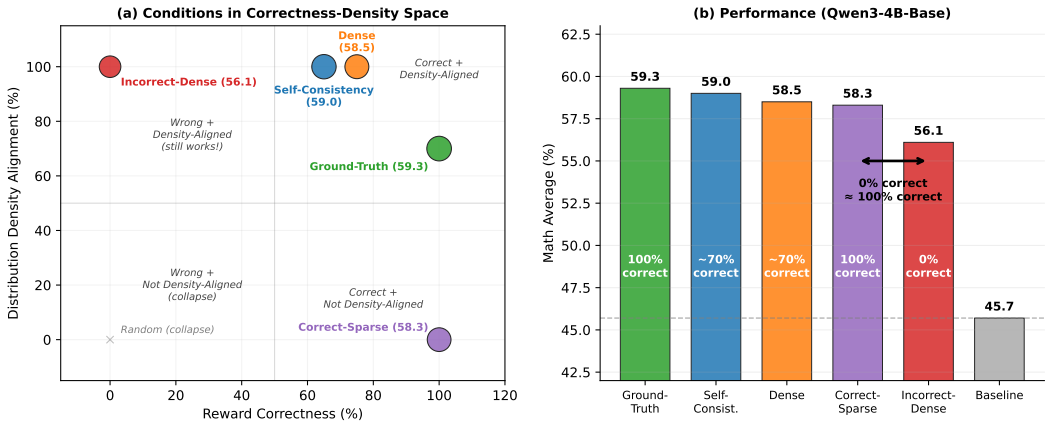


Figure 1: Core finding: reward *density alignment* with the model's output distribution is an independent mechanism driving self-supervised RL, beyond reward *correctness*. (a) Our experimental conditions span the correctness–density space. Bubble size indicates downstream math performance. (b) Performance comparison on Qwen3-4B-Base. Training with 0% correct but density-aligned rewards (Incorrect-Dense) still improves the model substantially, while random targets cause collapse.

is an approximate subset of those solvable by the base model. This suggests that RL redistributes probability mass toward existing paths rather than creating new reasoning capabilities.

One possible explanation for this robustness is that another mechanism guides the training in parallel with the reward signal. Press et al. revealed that such self-supervised training (i.e., entropy minimization) is not only success-driven, but also distribution-driven Press et al. [2024]. The model's performance still improves even with 0% correct labels. Further examination revealed that training changes the inner representation of post-training data, mapping it to an area close to similar pre-training data. Such proximity drives the model to act in a way acquired from pre-training, which adapts it for new tasks.

Given that self-supervised LLM RL training is essentially entropy minimization as well, we hypothesize that the distribution-driven mechanism also exists and that this explains why self-supervised training is robust to noisy rewards. In particular, in Group Relative Policy Optimization (GRPO) style training, the model samples multiple responses for each prompt, and the majority answer is positively rewarded whereas others are negatively rewarded. The diverse responses can be viewed as a probability distribution, and training aims to enhance the largest group (i.e., class). This is exactly the mechanism proposed by Zuo et al. [2025], Zhang et al. [2025c], where Zhang et al. framed it as semantic entropy optimization Zhang et al. [2025c].

We test this hypothesis with a design that isolates reward correctness from data distribution (Figure 1). Specifically, we pre-sample responses for the training prompts with the base model to obtain a landscape of its output distribution, and use it as reference density. Then we train the model with different types of rewards: 1) ground truth (main effects of success and distribution), 2) dynamic self-consistency (main effects of distribution + side effects of rewards), 3) reference density (main effects of distribution + side effects of rewards), 4) correct but sparse (full effects of success), 5) incorrect but dense (full effects of distribution), and 6) random and sparse (no effects of distribution or success). Results suggest that both the density and reward effects exist, with the effect of success slightly stronger than that of distribution. Our work proposes and confirms that distribution-driven processes indeed exist in LLM RL training. Our contributions are three-fold:

- We propose a hypothesis that LLM RL training is not only success-driven, but also a distribution-driven process.
- We confirm this hypothesis with experimental results showing that distribution-driven training can achieve comparable model performance to success-driven training.
- Our findings deepen the understanding of why RLVR works, informing future reward design to consider density alignment alongside correctness.

2 Related Work

Entropy minimization training (as a type of unsupervised learning) has been found to enhance model performance for a long time Lee [2013], Wang et al. [2021], Niu et al. [2023], Press et al. [2024]. Lee proposed a semi-supervised learning approach where labeled data were trained via standard supervised learning whereas unlabeled data were trained to minimize the model's output entropy Lee [2013]. Analysis showed that entropy minimization has a clustering effect, reducing within-group distance and increasing between-group distance, resulting in clearer separation boundaries. Wang et al. proposed an entropy minimization approach (TENT) for test-time adaptation, where a probability model was trained to minimize the entropy of its own outputs towards the most probable class on new data Wang et al. [2021]. Analysis showed that TENT better shapes the model's inner features towards the oracle than other approaches. Niu et al. found that filtering noisy samples and encouraging model weights to settle at a flat minimum could effectively stabilize entropy minimization test-time adaptation Niu et al. [2023]. Press et al. dug further into the underlying mechanism of entropy minimization and discovered that such training changes the model's inner representation of new data, mapping them closer to similar training data and thus generating similar outputs Press et al. [2024]. More importantly, the model's performance still improves even when 100% of labels are incorrect, suggesting that entropy minimization is not entirely success-driven. To summarize, entropy minimization has been shown to effectively adapt the model for new tasks, operating mainly through reshaping the model's inner representation of new data.

Self-supervised RL for LLMs has been explored by a considerable number of recent studies as RLVR has become one of the most active areas Zuo et al. [2025], Zhang et al. [2025c,b], Agarwal et al. [2025]. Researchers have investigated various types of rewards, such as self-consistency Zuo et al. [2025], Zhang et al. [2025c], internal confidence Agarwal et al. [2025], self-generated marks Yuan et al. [2024], Wu et al. [2025], and cross-view rewards Zhang et al. [2026]. In particular, test-time reinforcement learning (TTRL) positively rewards the majority group (in terms of the final answer) and negatively rewards all other groups, achieving comparable performance to ground-truth rewards. Entropy Minimized Policy Optimization (EMPO) applies a similar rewarding strategy, but weights the rewards relative to group size, effectively enhancing the majority class. Agarwal et al. proposed multiple entropy-based objectives for RL training, such as token-level entropy, sequence-level entropy, and token logits Agarwal et al. [2025]. Zhang et al. proposed to enhance label quality by referencing a second view, such as asking superficially different but fundamentally similar questions to the same model, or asking the same question to a different model Zhang et al. [2026]. By comparing the answers, it is more likely to obtain the correct label. Put together, prior work has found that enhancing self-consistency can effectively improve model performance. On the other hand, it is widely believed that model performance improves because it is trained towards the true label. Here, we show that training the model towards the dense region is another mechanism that can improve model performance.

3 Methods

3.1 Preliminary: Group Relative Policy Optimization

Group Relative Policy Optimization (GRPO) Shao et al. [2024], Guo et al. [2025a] is a policy gradient method widely adopted in recent LLM post-training. Given a prompt q, GRPO samples a group of G responses $\{o_1, \ldots, o_G\}$ from the current policy π_θ and assigns each response a scalar reward r_i via a reward function. The advantage of each response is computed by normalizing rewards within the group:

$$A_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)} \tag{1}$$

The policy is then updated by maximizing the clipped surrogate objective:

$$\mathcal{J}_{\text{GRPO}} = \mathbb{E}_{q\sim\mathcal{D},\, \{o_i\}\sim\pi_\theta(\cdot|q)} \left[\frac{1}{G} \sum_{i=1}^{G} \min\left(\rho_i A_i,\ \text{clip}(\rho_i, 1-\epsilon, 1+\epsilon) A_i\right) \right] \tag{2}$$

where $\rho_i = \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}$ is the importance sampling ratio between the current and old policy, and ϵ is the clipping threshold for training stability. Following recent practice Zhang et al. [2025c], Zuo et al. [2025], we set the KL penalty coefficient to zero.

In standard RLVR, the reward is computed by comparing the response against a ground-truth answer: $r_i = 1$ if the extracted answer matches the ground truth, and $r_i = 0$ otherwise. This binary reward is the only external signal that enters the optimization — it determines which responses receive positive advantage and which receive negative advantage.

3.2 Isolating Success from Distribution

As stated in the introduction, we hypothesize that self-supervised GRPO training contains two separable learning signals: a success factor (reward correctness) and a distribution factor (reward density). To test this, we need experimental conditions that independently vary these two factors while holding the training algorithm constant.

The key challenge is that under normal self-supervised training, correctness and density are confounded — the majority cluster is often correct in well-pretrained models. This confound deepens as training progresses with self-consistency rewards: as the model sharpens toward its majority output, that output becomes increasingly correct, making the two factors ever harder to disentangle. To break this confound, we need an independent estimate of where the model's output distribution is dense, separate from whether that region happens to be correct. We obtain this by pre-computing a fixed density reference before training begins: we sample multiple responses from the frozen base model for every training prompt and cluster them by semantic equivalence. The largest cluster's representative answer is stored as the *density target* for that prompt.

Pre-computing from the frozen base model, rather than estimating density online during training, is essential for two reasons. First, it provides a stable reference that does not co-evolve with the policy — if we estimated density online, the density target would shift as the model learns, making it impossible to attribute improvement to density alignment versus some other training dynamic. Second, it allows us to identify prompts where the density target disagrees with the ground truth *before* training starts, enabling us to purposely combine or isolate the two factors.

3.3 Study Design

Our study includes six training conditions that systematically vary whether the reward target is correct, dense, both, or neither. All conditions use identical GRPO training; only the reward function differs. Table 1 summarizes the conditions and their properties.

Table 1: Training conditions. *Correct*: reward target matches ground truth. *Dense*: reward target aligns with the model's densest output region. Conditions marked with † are restricted to prompts where the density target disagrees with the ground truth.

Condition	Reward target	Correct	Dense
GROUND-TRUTH	Ground-truth answer	✓	Partial
SELF-CONSISTENCY	Online majority vote	Partial	✓
DENSE	Pre-computed density target	Partial	✓
CORRECT-SPARSE†	Ground truth on conflict prompts	✓	×
INCORRECT-DENSE†	Density target on conflict prompts	×	✓
RANDOM	Random non-majority cluster	×	×

Ground-Truth. The standard RLVR baseline: $r_i = 1$ if the response matches the ground-truth answer. This provides full correctness signal and partial density alignment (since the correct answer often coincides with the majority cluster).

Self-Consistency. Replicates the EMPO/TTRL setup: at each training step, the G sampled responses are clustered online, and $r_i = 1$ if the response belongs to the largest cluster. Correctness and density are naturally entangled.

Dense. Rewards responses matching the pre-computed density target (the frozen reference model's majority answer). Unlike Self-Consistency, the target does not shift during training. The density target is correct on roughly 40%–70% of prompts depending on difficulty.

Correct-Sparse. Rewards the correct answer on conflict prompts only (where density target $\neq$ ground truth), with reward set to 0 on non-conflict prompts. This isolates the success mechanism from the distribution mechanism.

Incorrect-Dense. Rewards the density target on conflict prompts only, with reward set to 0 on non-conflict prompts. This isolates the distribution mechanism from the success mechanism — the reward has zero ground-truth accuracy by construction.

Random. Rewards responses matching a randomly selected non-majority cluster answer. This provides neither correctness nor density alignment and serves as the negative control.

4 Experiments

4.1 Experimental Setup

Models and data. We conduct experiments on Qwen3-4B-Base Yang et al. [2025] and Llama-3.2-3B-Instruct Grattafiori et al. [2024], which differ in architecture, scale, and whether instruction tuning has been applied. We use 20K prompts randomly sampled from NuminaMath-CoT Team [2024], following the same data source as EMPO Zhang et al. [2025c] and TTRL Zuo et al. [2025] for comparability.

Training. All conditions use GRPO Shao et al. [2024] with identical hyperparameters: 8 rollouts per prompt, global batch size of 64, learning rate 5×10^{-7}, and no KL penalty. We train for up to 300 steps with a maximum generation length of 3072 tokens, selecting the best checkpoint by validation performance. Responses shorter than 128 tokens are filtered out to reduce noise and reward hacking, following prior work showing that filtering low-quality responses stabilizes training Zhang et al. [2025c], Niu et al. [2023]. Temperature is 1.0 for Qwen and 0.6 for Llama during training (we tried both 1.0 and 0.6 for Llama; 0.6 worked slightly better). We use the Verl framework ByteDance Seed Team [2025] for training.[1] Full hyperparameter details are in Appendix A.

Density reference. For each training prompt, we sample 16 responses from the frozen base model and cluster by exact answer string equivalence. The largest cluster's answer becomes the density target. The density target agrees with ground truth on $\sim 52\%$ of prompts (Qwen) and $\sim 29\%$ (Llama), leaving substantial conflict subsets for the isolation conditions. This one-time pre-computation takes approximately 6 hours on 2×A100 80GB GPUs.

Evaluation. We evaluate on seven benchmarks: AMC23, AIME24, MATH-500, GSM8K (math), MMLU-Pro, GPQA-Diamond (knowledge), and LiveCodeBench (code). We report avg@16 accuracy. Best checkpoint is selected by AMC23 validation performance.

4.2 Main Results

Table 2 presents the main results across all conditions and benchmarks. We discuss findings revealing the impacts of success and distribution factors on model performance.

Density-aligned training matches ground-truth supervision on Qwen. On Qwen3-4B-Base, Dense (58.5), Self-Consistency (59.0), and Ground-Truth (59.3) achieve nearly identical math averages. The density target is correct on only 40%–70% of prompts, yet training toward it produces performance indistinguishable from training with verified correct labels. This holds consistently across all four math benchmarks, with Dense even slightly exceeding Ground-Truth on AMC23 (54.1 vs. 52.2) and GSM8K (92.9 vs. 92.6). Notably, the fixed pre-computed density target (Dense) performs comparably to the online majority vote (Self-Consistency), indicating that the base model's initial density landscape is already sufficiently informative.

Training toward incorrect-but-dense targets still improves the model. Incorrect-Dense achieves 56.1 math avg on Qwen (vs. 45.6 base), a substantial improvement despite exclusively rewarding

[1] Code and data will be released upon acceptance.

Table 2: Main results across seven benchmarks. All scores are avg@16 accuracy (%). Math Avg. is the mean of AMC23, AIME24, MATH-500, and GSM8K.

	AMC23	AIME24	MATH500	GSM8K	Math Avg.	MMLU Pro	GPQA Diamond	LiveCode Bench
Qwen3-4B-Base								
Baseline	43.4	8.3	62.6	68.3	45.6	28.6	24.4	17.4
Ground-Truth	52.2	14.2	78.1	92.6	59.3	50.3	40.1	27.1
Self-Consistency	51.3	14.2	79.3	91.3	59.0	48.1	39.9	14.3
Dense	54.1	11.3	75.5	92.9	58.5	49.3	36.0	24.9
Correct-Sparse	56.9	15.4	71.9	89.1	58.3	37.8	32.6	26.2
Incorrect-Dense	52.5	8.8	72.0	91.0	56.1	46.5	40.8	28.2
Random	–	–	–	–	–	–	–	–
Llama-3.2-3B-Instruct								
Baseline	17.5	6.3	41.5	71.0	34.1	25.0	20.0	5.4
Ground-Truth	33.1	9.2	55.9	77.4	43.9	29.6	25.9	7.0
Self-Consistency	30.0	4.2	36.9	72.1	35.8	11.5	13.0	7.5
Dense	28.4	9.6	36.4	78.0	38.1	25.3	21.1	3.8
Correct-Sparse	30.3	7.9	46.3	84.0	42.1	29.4	21.9	6.9
Incorrect-Dense	18.4	10.8	39.3	61.9	32.6	13.6	18.9	5.8
Random	–	–	–	–	–	–	–	–

wrong answers on conflict prompts. On knowledge and code benchmarks, it slightly exceeds Ground-Truth on GPQA (40.8 vs. 40.1) and LiveCodeBench (28.2 vs. 27.1). Recall that this condition only trains on prompts where the density target is known to disagree with the ground truth — every rewarded response is incorrect by construction. The fact that the model still improves is the strongest evidence for the distribution mechanism: density alignment provides a genuine learning signal independent of reward correctness.

Density alignment provides broader transfer than correctness alone. On non-math benchmarks, density-aligned conditions substantially outperform Correct-Sparse: MMLU-Pro (46.5–49.3 vs. 37.8) and GPQA (36.0–40.8 vs. 32.6). Correct-Sparse achieves competitive math performance (58.3) but does not transfer as effectively to knowledge reasoning. This gap is notable because all conditions are trained on the same math prompts — the difference in transfer suggests that density alignment contributes a more general representational improvement beyond task-specific problem-solving skills.

The distribution mechanism is weaker but present on Llama. On Llama-3.2-3B-Instruct, Dense (38.1) improves over base (34.1) on math avg, confirming that density alignment provides a learning signal even for this weaker model. However, correctness-based conditions clearly dominate (Ground-Truth 43.9, Correct-Sparse 42.1). Incorrect-Dense (32.6) does not cause model collapse but does not improve performance either, suggesting that on weaker models the density signal from conflict prompts alone is insufficient. Self-Consistency (35.8) also barely improves over base and collapses on MMLU-Pro (11.5) and GPQA (13.0), consistent with prior findings that self-supervised RL is less effective on models with lower base capability. A detailed comparison of the two models' density landscapes (Appendix C, Table 4) shows that Llama's density targets match ground truth on only 34.7% of prompts versus 52.2% for Qwen, indicating that its dense regions are far less reliably correct.

Random targets cause collapse. The Random condition degrades performance on both models, confirming that the improvements observed in all other conditions reflect genuine learning signals rather than artifacts of the GRPO training procedure. Arbitrary reward targets that are neither correct nor density-aligned do not provide useful gradient direction.

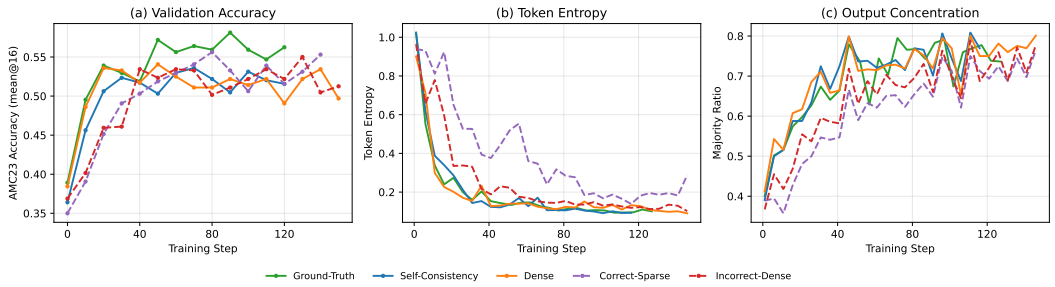


Figure 2: Training dynamics on Qwen3-4B-Base across conditions. (a) Validation accuracy on AMC23 (mean@16) evaluated every 10 steps. (b) Token-level entropy during training. (c) Output concentration measured as the fraction of responses belonging to the majority cluster. All effective conditions reduce entropy and increase output concentration, regardless of whether the reward target is correct.

4.3 Training Dynamics

Figure 2 shows training dynamics across conditions on Qwen3-4B-Base. Three patterns emerge. First, all conditions that improve downstream performance also reduce token entropy substantially (from $\sim$1.0 to 0.07–0.16) and increase output concentration (majority ratio from $\sim$0.4 to 0.7–0.8). This holds for Incorrect-Dense, which rewards wrong answers, just as strongly as for Ground-Truth. The shared signature of effective training is not learning correct answers but organizing the output distribution — concentrating probability mass into a coherent mode. Second, Correct-Sparse reduces entropy more slowly and to a lesser degree (final entropy 0.16 vs. 0.07–0.10 for density-aligned conditions), consistent with its sparse reward signal providing weaker gradient. Third, we also explored a Second-Largest condition that targets a non-dominant cluster; it achieves substantially less entropy reduction and output concentration than density-aligned conditions, confirming that alignment with the model's existing dense region specifically drives the mechanism (full results in Appendix F).

On Llama-3.2-3B-Instruct (Appendix), the dynamics differ qualitatively. The instruction-tuned model starts with low entropy ($\sim$0.12), leaving less room for further reduction. Correct-Sparse and Dense on Llama actually *increase* entropy ($+0.08$ and $+0.11$ respectively), suggesting that when the base model's dense regions are less reliable, training destabilizes rather than consolidates the output distribution. Notably, we initially trained Llama at temperature 1.0, which produced high entropy but little performance gain; switching to temperature 0.6 yielded lower entropy yet better performance. This suggests that the key process is distributional organization rather than entropy reduction per se — effective training can occur even without large entropy changes, provided the output distribution becomes more coherent.

5 Discussion

5.1 Why Does Density Alignment Work?

Our results demonstrate that training toward the model's dense output region improves performance even when the targets are incorrect (Figure 3). The Dense condition's reward target matches ground truth on only $\sim$65–73% of prompts, yet its downstream performance matches Ground-Truth (which uses 100% correct labels). The policy match rate — how often the model's majority output matches the pre-computed density target — increases from 0.39 to 0.79 over training (Figure 3c), confirming that the model converges toward its pre-existing dense mode rather than reducing entropy in an arbitrary direction. This aligns with findings that RL post-training surfaces reasoning patterns already embedded during pretraining rather than creating new ones Zhang et al. [2025c], Brown et al. [2024], Yue et al..

Density-aligned training exploits this structure by rewarding responses that align with the model's existing mode, sharpening the policy toward regions where pretraining already concentrated probability mass — the same mechanism Press et al. Press et al. [2024] identified in vision models. Critically,

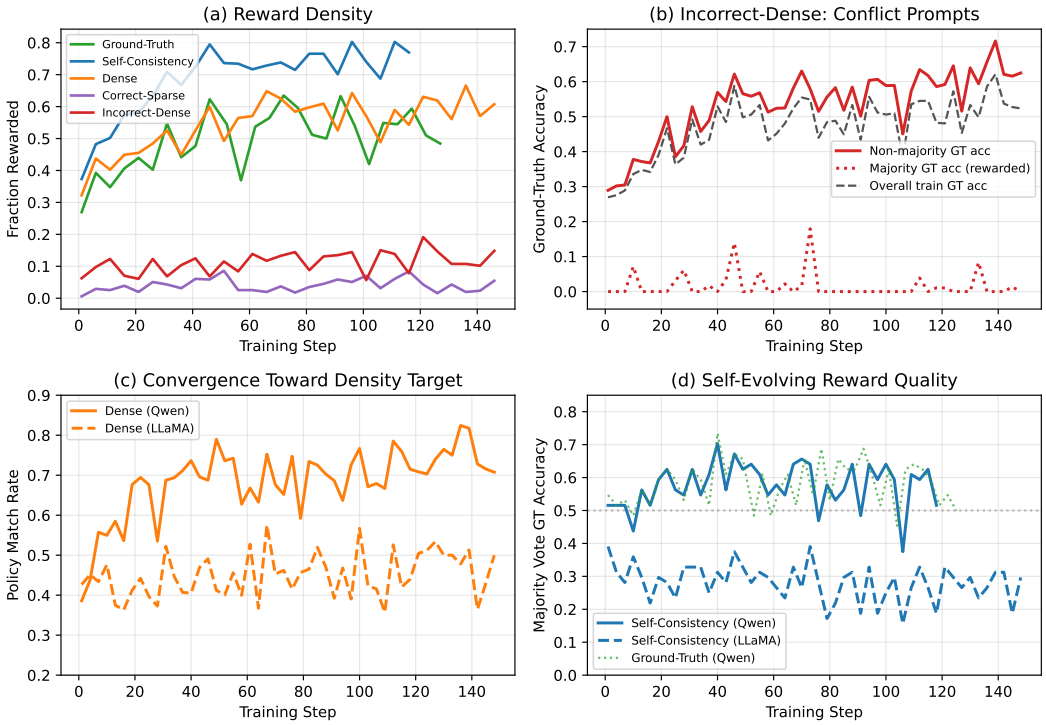


Figure 3: Analysis of training dynamics supporting the distribution mechanism (see Section 5). (a) Reward density (fraction of responses receiving positive reward) across conditions — density-aligned conditions reward 30–60% of responses while Correct-Sparse rewards only 3–10%. (b) GT accuracy breakdown on Incorrect-Dense conflict prompts. The reward target is the *reference* model's majority (which is incorrect by construction on conflict prompts), but the *current* model's majority may differ from the reference majority — hence the small non-zero GT accuracy. As training progresses, the current model's majority converges toward the (incorrect) reference target, driving majority GT accuracy toward zero. Meanwhile, non-majority responses become progressively more correct (0.29→0.59), indicating that distributional organization improves even unrewarded modes. (c) Policy match rate for the Dense condition: the model converges toward its pre-computed density target. (d) Self-evolving reward quality: majority vote accuracy improves on Qwen but degrades on Llama.

density alignment also determines gradient quality: Dense and Ground-Truth reward 30–60% of responses per step, while Correct-Sparse rewards only 3–10% (Figure 3a), producing $\sim 2\times$ lower gradient norms (0.16 vs. 0.32–0.36). The Random condition provides a critical control — it also reduces entropy, yet causes collapse, showing that only targets aligned with the model's pre-existing density landscape provide useful signal.

5.2 Two Mechanisms, Different Contributions

Our factorial design reveals that correctness and density contribute differently. Correct-Sparse achieves strong math performance (58.3 avg) but transfers poorly to knowledge benchmarks (MMLU-Pro 37.8, GPQA 32.6). Density-aligned conditions show the opposite: slightly lower math gains but substantially better transfer (MMLU-Pro 46.5–49.3, GPQA 36.0–40.8). Correctness teaches task-specific problem-solving while density alignment provides broader representational improvement that benefits reasoning across domains. This decomposition explains why TTRL Zuo et al. [2025] reported performance surpassing Maj@N (the training signal itself) — the density mechanism provides learning signal beyond what labels encode — and why EMPO Zhang et al. [2025c] and Agarwal et al. Agarwal et al. [2025] found that self-supervised RL with noisy rewards (40%–70% accuracy) achieves near-GT performance: even incorrect rewards contribute useful gradient through density alignment.

5.3 When Does the Distribution Mechanism Fail?

Incorrect-Dense pushes the distribution mechanism to its logical extreme: on Qwen, with *zero* correct reward, AMC23 accuracy still improves from 0.37 to 0.56 ($+19$ points) as entropy drops from 0.96 to 0.09. On conflict prompts, the reward target is the reference model's majority answer (which is incorrect by construction), but the current model's majority may initially differ from the reference majority — hence a small non-zero GT accuracy early in training. As the model converges toward the reference target, majority GT accuracy drops to near zero, yet non-majority responses become more correct ($0.29 \rightarrow 0.59$; Figure 3b), suggesting that distribution organization creates coherent minority modes even under adversarial reward. However, on Llama (initial entropy 0.12), Incorrect-Dense *degrades* performance ($0.24 \rightarrow 0.17$), suggesting that the mechanism requires sufficient room for the output distribution to reorganize.

This boundary condition is consistent with Agarwal et al. Agarwal et al. [2025], who reported that entropy minimization improves Qwen but not Llama, and explains why larger models benefit more from self-supervised RL Zuo et al. [2025]: better-organized pretraining representations yield more informative dense regions. For Self-Consistency, the majority vote's GT accuracy improves on Qwen ($0.52 \rightarrow 0.63$) but degrades on Llama ($0.39 \rightarrow 0.13$; Figure 3d), explaining why self-supervised RL benefits capable models disproportionately. Practically, this suggests a diagnostic before attempting self-supervised RL: measure the correlation between the model's majority cluster and ground-truth correctness. If this correlation is low, labeled rewards may be necessary.

5.4 Implications for RLVR Practice

Our findings suggest several practical considerations for the RLVR community:

1. **Reward accuracy is not the only relevant dimension.** A reward signal can be useful even with low accuracy, provided it is density-aligned. This reframes reward design: practitioners should consider both correctness and density alignment when constructing reward functions.
2. **Fixed density targets can replace online majority voting.** Our Dense condition (pre-computed from the frozen base model) performs comparably to Self-Consistency (online majority vote). This eliminates the need for expensive online clustering during training, reducing computational cost while preserving the distribution mechanism.
3. **Combining both mechanisms may yield stronger training.** Since correctness and density contribute differently (task-specific vs. broad transfer), explicitly combining both signals — e.g., correctness rewards augmented with a density bonus — could produce models that are both accurate on the trained domain and transfer well to new tasks.
4. **Self-supervised RL success is predictable.** The distribution mechanism's effectiveness depends on base model capability. Before investing in self-supervised RL, practitioners can estimate its likely benefit by checking whether the model's majority outputs correlate with correctness.

6 Conclusion

We presented evidence that self-supervised reinforcement learning in LLMs is driven by two distinct mechanisms: a *success mechanism* (learning from correct rewards) and a *distribution mechanism* (aligning with the model's pre-existing output density). Through a controlled factorial design that isolates these mechanisms, we showed that density-aligned rewards — even with 0% correctness — substantially improve model performance, while correct but density-misaligned rewards yield weaker gains. Both mechanisms contribute to the effectiveness of standard RLVR, but the distribution mechanism has been previously unrecognized.

Our findings reframe the understanding of why self-supervised RL works: it succeeds not primarily because the model's self-generated rewards are correct, but because they are structurally aligned with the model's output distribution. This explains the robustness of methods like TTRL and EMPO to noisy reward signals, and connects to concurrent findings that RL redistributes rather than creates reasoning capabilities. We hope this work informs future reward design by highlighting density alignment as a first-class consideration alongside correctness.

References

Shivam Agarwal, Zimin Zhang, Lifan Yuan, Jiawei Han, and Hao Peng. The unreasonable effectiveness of entropy minimization in LLM reasoning. In *Advances in Neural Information Processing Systems*, 2025. URL https://mlanthology.org/neurips/2025/agarwal2025neurips-unreasonable/.

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL https://arxiv.org/abs/2407.21787.

ByteDance Seed Team. verl: Volcano engine reinforcement learning for llms, 2025. URL https://github.com/volcengine/verl.

Kanishk Gandhi, Ayush K. Chakravarthy, Anikait Singh, Nathan Lile, and Noah Goodman. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective STars. In *Second Conference on Language Modeling*, 2025. URL https://openreview.net/forum?id=QGJ9ttXLTy.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025a. doi: 10.1038/s41586-025-09422-z. URL https://www.nature.com/articles/s41586-025-09422-z.

Yiran Guo, Lijie Xu, Jie Liu, Dan Ye, and Shuang Qiu. Segment policy optimization: Effective segment-level credit assignment in RL for large language models. In *Advances in Neural Information Processing Systems*, volume 38, pages 114399–114431, 2025b. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/a6536243037d1e32c20de85137d478da-Paper-Conference.pdf.

Leslie Pack Kaelbling, Michael L. Littman, and Andrew W. Moore. Reinforcement learning: A survey. *Journal of Artificial Intelligence Research*, 4:237–285, 1996. doi: 10.1613/jair.301.

Dong-Hyun Lee. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *ICML Workshop on Challenges in Representation Learning*, 2013.

Michael L. Littman. Reinforcement learning improves behaviour from evaluative feedback. *Nature*, 521(7553):445–451, 2015. doi: 10.1038/nature14540. URL https://www.nature.com/articles/nature14540.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236. URL https://www.nature.com/articles/nature14236.

Shuaicheng Niu, Jiaxiang Wu, Yifan Zhang, Zhiquan Wen, Yaofo Chen, Peilin Zhao, and Mingkui Tan. Towards stable test-time adaptation in dynamic wild world. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=g2YraF75Tj.

Ori Press, Ravid Shwartz-Ziv, Yann LeCun, and Matthias Bethge. The entropy enigma: Success and failure of entropy minimization. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 41064–41085. PMLR, 2024. URL https://proceedings.mlr.press/v235/press24a.html.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.

Shuaijie She, Junxiao Liu, Yifeng Liu, Jiajun Chen, Xin Huang, and Shujian Huang. R-PRM: Reasoning-driven process reward modeling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 13438–13451, Suzhou, China, nov 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.679. URL https://aclanthology.org/2025.emnlp-main.679/.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. doi: 10.1038/nature16961. URL https://www.nature.com/articles/nature16961.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Adaptive Computation and Machine Learning. MIT Press, Cambridge, MA, 1998. ISBN 9780262193986.

AI-MO Team. Numinamath 1.5: A large-scale dataset for mathematical reasoning, 2024. URL https://huggingface.co/datasets/AI-MO/NuminaMath-CoT.

Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=uXl3bZLkr3c.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9426–9439, Bangkok, Thailand, aug 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.510. URL https://aclanthology.org/2024.acl-long.510/.

Tianhao Wu, Weizhe Yuan, Olga Golovneva, Jing Xu, Yuandong Tian, Jiantao Jiao, Jason E. Weston, and Sainbayar Sukhbaatar. Meta-rewarding language models: Self-improving alignment with LLM-as-a-meta-judge. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 11537–11554, Suzhou, China, nov 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.583. URL https://aclanthology.org/2025.emnlp-main.583/.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. Free process rewards without process labels. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 73511–73525. PMLR, 2025. URL https://proceedings.mlr.press/v267/yuan25c.html.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason E. Weston. Self-rewarding language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 57905–57923. PMLR, 2024. URL https://proceedings.mlr.press/v235/yuan24d.html.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Kaiyan Zhang, Yuxin Zuo, Bingxiang He, Youbang Sun, Runze Liu, Che Jiang, Yuchen Fan, Kai Tian, Guoli Jia, Pengfei Li, et al. A survey of reinforcement learning for large reasoning models. *arXiv preprint arXiv:2509.08827*, 2025a. URL https://arxiv.org/abs/2509.08827.

Kongcheng Zhang, Qi Yao, Shunyu Liu, Yingjie Wang, Baisheng Lai, Jieping Ye, Mingli Song, and Dacheng Tao. Consistent paths lead to truth: Self-rewarding reinforcement learning for LLM reasoning. In *Advances in Neural Information Processing Systems*, volume 38, pages 59849–59887, 2025b. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/565f995643da6329cec701f26f8579f5-Paper-Conference.pdf.

Qingyang Zhang, Haitao Wu, Changqing Zhang, Peilin Zhao, and Yatao Bian. Right Question Is Already Half the Answer: Fully Unsupervised LLM Reasoning Incentivization. In *Advances in Neural Information Processing Systems*, 2025c. URL https://mlanthology.org/neurips/2025/zhang2025neurips-right/.

Zizhuo Zhang, Jianing Zhu, Xinmu Ge, Zihua Zhao, Zhanke Zhou, Xuan Li, Xiao Feng, Jiangchao Yao, and Bo Han. Co-rewarding: Stable self-supervised RL for eliciting reasoning in large language models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=fDk95XPsCU.

Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, Haozhan Li, Yuchen Zhang, Xinwei Long, Ermo Hua, Biqing Qi, Youbang Sun, Zhiyuan Ma, Lifan Yuan, Ning Ding, and Bowen Zhou. TTRL: Test-time reinforcement learning. In *Advances in Neural Information Processing Systems*, 2025. URL https://mlanthology.org/neurips/2025/zuo2025neurips-ttrl/.

A Training Details

All experiments use the GRPO algorithm Guo et al. [2025a] with the following shared configuration. Training is conducted on 2×A100 80GB GPUs using the Verl framework ByteDance Seed Team [2025]. Each condition takes 5.5–13 hours (Qwen) or 6.4–10.2 hours (Llama). Density pre-computation requires approximately 6 hours on 2×A100 80GB GPUs. Total compute across all 11 conditions (including density pre-computation) is approximately 200–250 GPU-hours. We run 2–5 seeds per condition to verify stability; reported results use the best checkpoint selected by AMC23 validation accuracy (evaluated every 10 steps).

Temperature selection for Llama. We experimented with both temperature 1.0 and 0.6 for Llama training. Temperature 0.6 yielded slightly better results, likely because Llama-3.2-3B-Instruct is already instruction-tuned and has a narrower output distribution; temperature 1.0 produces overly diffuse samples that dilute the reward signal. All Llama results reported in this paper use temperature 0.6.

B Dataset Details

Training data. We use the same 20,000 math problems from NuminaMath-CoT Team [2024] as released by EMPO Zhang et al. [2025c]. For each prompt, we pre-sample 16 responses from the base model (Qwen3-4B-Base or LLaMA-3.2-3B-Instruct) at temperature 1.0 to construct the reference density landscape. Answers are extracted via regex matching of `\boxed{}` patterns and clustered by exact string equivalence. The density target for each prompt is defined as the answer of the largest cluster.

Training data statistics. Across the 20,000 prompts:

- Mean number of answer clusters per prompt: 10.3 (out of 16 responses)
- Mean largest cluster size: 5.4/16 (density ratio = 0.34)
- Prompts with clear majority (>50% in one cluster): 17.8%
- Density target matches ground truth: 52.2% of prompts
- Conflict prompts (density $\neq$ ground truth): 47.8%

Table 3: Training hyperparameters shared across all conditions.

Parameter	Value
Model	
Base model (Qwen)	Qwen3-4B-Base
Base model (LLaMA)	LLaMA-3.2-3B-Instruct
Generation	
Rollout group size (n)	8
Max prompt length	1024 tokens
Max response length	3072 tokens
Min response length filter	128 tokens
Generation batch size	64 prompts
Temperature	1.0 (Qwen) / 0.6 (Llama)
Optimization	
Policy loss	GSPO
Advantage estimator	GRPO
Learning rate	5×10^{-7}
LR warmup ratio	0.1
Clip ratio (low / high)	3×10^{-4} / 4×10^{-4}
Clip ratio c	10.0
KL coefficient	0.0 (no KL penalty)
Gradient checkpointing	Enabled
Training Schedule	
Training batch size	64 prompts $\times$ 8 responses = 512
Total training steps	300 (Qwen), 180 (LLaMA)
PPO epochs per batch	1
Checkpoint selection	Top-k by AMC23 val accuracy (eval every 10 steps)

Evaluation benchmarks. We evaluate on seven benchmarks spanning math reasoning, knowledge reasoning, and code:

- **AMC23** (40 problems): AMC competition 2023. Used as validation during training.
- **AIME24** (30 problems): AIME competition 2024.
- **MATH500** (500 problems): Subset of the MATH benchmark.
- **GSM8K** (1,319 problems): Grade school math word problems.
- **MMLU-Pro** (12,032 problems): Multi-domain knowledge reasoning with 10-choice format.
- **GPQA Diamond** (198 problems): Graduate-level science QA.
- **LiveCodeBench** (1,055 problems): Code generation from recent programming contests.

All evaluations use mean@16: we sample 16 responses per problem and report the average accuracy across all samples. For benchmarks containing more than 100 prompts, we randomly select 100 prompts to reduce validation cost.

C Density Computation Details

The reference density is pre-computed once from the frozen base model before training begins. The procedure is:

1. **Response sampling.** For each of the 20,000 training prompts, generate 16 responses from the base model at temperature 1.0 with max length 3072 tokens.
2. **Answer extraction.** Extract the final answer from each response using regex matching of `\boxed{...}` patterns. Responses without a valid boxed answer are assigned to a singleton cluster.
3. **Clustering.** Group responses by exact answer string equivalence. This produces k clusters $\{C_1, C_2, \ldots, C_k\}$ per prompt, where k ranges from 1 to 16.

4. **Density target selection.** The density target is the answer of the largest cluster: $a^* = \arg\max_a |C_a|$. Ties are broken arbitrarily.
5. **Conflict identification.** A prompt is labeled as "conflict" if the density target a^* differs from the ground-truth answer. Approximately 47.8% of prompts are conflict prompts.

During training, the reward for each condition is assigned as follows:

- **Ground-Truth**: reward = 1 if the response's extracted answer matches the ground-truth answer.
- **Self-Consistency**: reward = 1 if the response's answer matches the online majority vote (recomputed each batch from the current policy's 8 responses).
- **Dense**: reward = 1 if the response's answer matches the pre-computed density target a^*.
- **Correct-Sparse**: reward = 1 if the response's answer matches ground truth, but only on conflict prompts (non-conflict prompts receive zero reward).
- **Incorrect-Dense**: reward = 1 if the response's answer matches the density target a^*, but only on conflict prompts where $a^* \neq$ ground truth (non-conflict prompts receive zero reward).
- **Random**: reward = 1 if the response's answer matches a randomly selected cluster's answer (not the largest).

Density landscape statistics. Table 4 summarizes the pre-computed density landscape for both models. Notably, while both models have similar density ratios ($\sim$0.34), Llama has a substantially lower GT match rate (29.1% vs. 52.2%), meaning its dense regions are far less reliably correct. This explains why the distribution mechanism is weaker for Llama (Section 5.3): sharpening toward density targets that are correct only 29% of the time provides a much weaker learning signal.

Table 4: Pre-computed density landscape statistics for both models (16 responses per prompt, 20K prompts). Llama is sampled at both temperature 1.0 (matching Qwen) and 0.6 (used in training).

Statistic	**Qwen3-4B-Base (temp 1.0)**	**Llama (temp 1.0)**	**Llama (temp 0.6)**
Mean largest cluster size	5.4 / 16	5.5 / 16	6.9 / 16
Mean density ratio	0.338	0.345	0.432
Mean number of clusters	10.3	9.2	7.8
Density target matches GT	52.2%	29.1%	34.7%
Prompts with clear majority (>50%)	17.8%	17.4%	30.7%
Conflict prompts (target $\neq$ GT)	47.8%	70.9%	65.3%

D LLaMA Training Dynamics

Figure 4 shows the training dynamics for LLaMA-3.2-3B-Instruct across all conditions. Compared to Qwen (Figure 2 in the main text), LLaMA starts with much lower entropy (0.12 vs. 0.96) because it is already instruction-tuned. Key differences:

- **Entropy**: LLaMA starts near-converged (0.12) with little room for further reduction. Qwen starts high (0.96) and drops to 0.09.
- **Reward density**: LLaMA achieves higher initial reward density due to its already-organized outputs, but the gap between conditions is smaller.
- **Incorrect-Dense**: On LLaMA, this condition *degrades* performance (val accuracy drops from 0.24 to 0.17), confirming that the distribution mechanism requires initial disorganization to operate.
- **Self-Consistency**: On LLaMA, the majority vote accuracy decreases over training (0.39 $\rightarrow$ 0.13), indicating that the self-evolving reward becomes self-reinforcing in the wrong direction for instruction-tuned models.

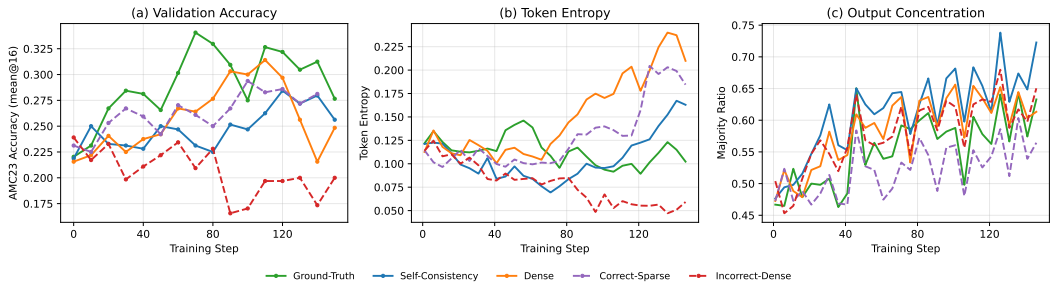


Figure 4: Training dynamics for LLaMA-3.2-3B-Instruct. Unlike Qwen (base model), LLaMA starts with low entropy and already-organized outputs. The distribution mechanism provides minimal benefit, and Incorrect-Dense actively degrades performance.

E Prompt Templates

Training prompt format. All training prompts follow the single-turn format:

```
{question} Let's think step by step and output
the final answer within \boxed{}.
```

Answer extraction. We extract the final answer using regex matching of the last `\boxed{...}` occurrence in the response. For nested braces, we use a balanced-brace parser. Responses without a valid boxed answer are treated as incorrect for reward computation and assigned to singleton clusters for density computation.

Evaluation. The same prompt format is used for evaluation. We sample 16 responses per problem at temperature 1.0 and report mean@16 (the fraction of responses that are correct, averaged across problems).

F Second-Largest Cluster Condition

To further test whether the distribution mechanism requires alignment with the *dominant* cluster specifically, we run an additional condition on Qwen3-4B-Base: SECOND-LARGEST, which rewards responses matching the second-largest cluster's answer from the reference model's pre-computed output distribution rather than the largest. This target is still structured (it reflects a coherent subset of the reference model's outputs) but is not the dominant mode.

Table 5 shows the results. Second-Largest improves over baseline on math benchmarks (+8.6 avg) but substantially less than Dense (+12.9 avg). Critically, it fails to transfer to non-math benchmarks (MMLU-Pro, GPQA), unlike density-aligned conditions. Training dynamics (Figure 2) show that Second-Largest achieves the least entropy reduction (final 0.28 vs. 0.07–0.10 for density-aligned conditions) and lowest output concentration (majority ratio 0.49 vs. 0.70–0.80). This confirms that the distribution mechanism is not simply "reward any consistent target" but specifically requires alignment with the model's dominant output mode.

Table 5: Benchmark results for the Second-Largest condition (Qwen3-4B-Base), compared to Dense and Baseline.

	AMC23	AIME24	MATH500	GSM8K	Math Avg	MMLU-Pro	GPQA	LCB
Baseline	43.4	8.3	62.6	68.3	45.6	28.6	24.4	17.4
Second-Largest	51.6	12.1	71.5	81.4	54.2	28.0	19.4	6.5
Dense	54.1	11.3	75.5	92.9	58.5	49.3	36.0	24.9

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction clearly state three contributions: (1) proposing the distribution-driven hypothesis, (2) confirming it experimentally, and (3) advancing understanding of LLM RL mechanisms. The experimental results in Section 4 directly support these claims.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 5.3 discusses when the distribution mechanism fails (weaker models, instruction-tuned models with low initial entropy). The paper is tested on only two model families at 3B–4B scale with math-only training data, which is acknowledged through the experimental scope.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: This is an empirical paper. It does not include theoretical results, theorems, or proofs.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Full training hyperparameters are provided in Appendix A (Table 2), density computation procedure in Appendix C, reward definitions for all six conditions in Appendix C, dataset details in Appendix B, and prompt templates in Appendix E. The training framework (Verl) and all model identifiers are specified.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: Code and data will be released upon acceptance to preserve anonymity during review. The training data (NuminaMath-CoT) is publicly available, and all implementation details are provided in the appendix to enable independent reproduction.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 4.1 provides the experimental setup including models, training data, hyperparameters, and evaluation protocol. Appendix A provides the full hyperparameter table, and Appendix B details the dataset statistics and evaluation benchmarks.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

Justification: We report mean@16 accuracy (averaging over 16 sampled responses per problem), which provides variance reduction at evaluation time. We run 2–5 seeds per condition to verify stability; however, we do not report error bars across seeds in the main table. The consistent patterns across two model families and seven benchmarks support the robustness of our findings.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix A specifies the hardware (2×A100 80GB GPUs per condition). Each condition takes 5.5–13 hours (Qwen) or 6.4–10.2 hours (Llama). Density pre-computation takes approximately 6 hours. Total compute across all experiments is approximately 200–250 GPU-hours.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: This research studies training dynamics of publicly available language models on publicly available math datasets. It does not involve human subjects, private data, or dual-use concerns.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: This work is foundational research studying the mechanisms of RL training in language models. It does not introduce new capabilities, deployable systems, or applications with direct societal impact beyond advancing scientific understanding of existing training methods.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: This paper does not release new models or datasets. The trained models are small-scale (3B–4B parameters) research checkpoints for mathematical reasoning and do not pose meaningful misuse risk beyond what the publicly available base models already provide.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.

- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: All models (Qwen3-4B-Base, Llama-3.2-3B-Instruct), datasets (NuminaMath-CoT), evaluation benchmarks (MATH, GSM8K, MMLU-Pro, GPQA, LiveCodeBench, AMC23, AIME24), and frameworks (Verl) are properly cited and used under their respective licenses.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: This paper does not release new assets at submission time. Code and data will be released upon acceptance, as noted in the paper.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: This paper does not involve crowdsourcing or research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: This paper does not involve human subjects research.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs are the subject of study in this paper (we train and evaluate them), which is standard ML research methodology. No LLM was used as a non-standard component in the research pipeline itself.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.